P1

Del C al ensamblador y a la predicción

Parte I · RA1 · 2,5 h

Universidad de Almería · Edición de 27 de septiembre de 2026

Documento: Guion de práctica para el alumnado
Asignatura: Ingeniería Inversa y Análisis Forense Digital
Uso: Laboratorio docente en entorno controlado y sobre las muestras indicadas
Alumno/a: _____ Grupo: _____

Índice

1. Mapa de trabajo **2**

2. Preparación **2**

3. Trabajo en sesión (150 minutos) **2**

4. Entregable **2**

5. Evaluación **2**

6. Teoría necesaria para realizar la práctica **3**
6.1. Del código C a instrucciones . . . 3
6.2. Convención System V AMD64 . . . 3
6.3. Dirección y contenido . . . 3

7. Software requerido y comprobación previa **3**

8. Guía detallada de realización **3**
8.1. 1. Preparar y registrar las muestras . . . 3
8.2. 2. Predecir antes de mirar el ensamblador . . . 4
8.3. 3. Obtener un desensamblado estable . . . 4
8.4. 4. Reconstruir el flujo . . . 4
8.5. 5. Comparar `-O0` y `-O2` . . . 4
8.6. 6. Contrastar las predicciones . . . 4

9. Controles de calidad y errores frecuentes **5**

10. Estructura recomendada de la entrega **5**

1 Mapa de trabajo

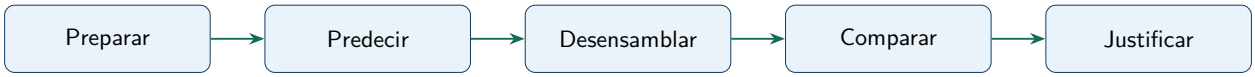


Figura 1: Secuencia de trabajo y puntos de control de la práctica.

El diagrama resume el orden recomendado. Cada transición debe quedar respaldada por una anotación, salida o fichero verificable en la entrega.

2,5 h · RA1 · 5 % de la nota de Parte I. Guion completo en LaTeX. Rúbrica única en criterios de evaluación.

2 Preparación

Desde la raíz del repositorio, en Linux x86-64:

```
make -C muestras/build x86
objdump -d -M intel --disassemble=funcion_bucle muestras/build/salida/ejemplos-x86-dbg
objdump -d -M intel --disassemble=funcion_bucle muestras/build/salida/ejemplos-x86-opt
```

Se permite leer `ejemplos.c`: es un ejercicio de relación fuente/máquina. No hacen falta compiladores cruzados ni QEMU para el núcleo.

3 Trabajo en sesión (150 minutos)

1. Preparación y predicción manual (20 min): resultado de `funcion_bucle(0)`, `(1)`, `(10)`, y argumentos de `funcion_llamada_con_argumentos(1,2,3)`.
2. Lectura de `-O0` (55 min): anotar una llamada, el condicional y el bucle, registros de argumentos/retorno y accesos a pila; indicar dónde se comprueba la condición.
3. Contraste `-O2` (35 min): localizar dos transformaciones y explicar por qué no hay correspondencia uno a uno entre C e instrucciones. No exigir saltos eliminados.
4. Memoria y tipos (25 min): distinguir dirección y contenido; justificar anchura y comparación con/sin signo en dos instrucciones elegidas.
5. Revisión individual (15 min): explicar un fragmento nuevo de igual dificultad.

4 Entregable

Tabla patrón→instrucción→efecto, tres predicciones justificadas, comparación de optimización y versión/comando de compilación. Dos o tres páginas más extractos bastan. La sintaxis utilizada será Intel y las direcciones procederán del binario entregado.

Ampliación: AArch64 con `aarch64-linux-gnu-objdump`, sin peso adicional. Para ejecutar binarios dinámicos con QEMU se necesita un sysroot compatible (`-L`); el `objdump` nativo no tiene necesariamente habilitadas todas las arquitecturas.

5 Evaluación

Se aplica la rúbrica común: corrección técnica 40 %, evidencia reproducible 25 %, interpretación y límites 25 %, claridad 10 %. Todos los criterios se aplican al objetivo concreto de esta práctica; las ampliaciones no son necesarias para la máxima nota.

6 Teoría necesaria para realizar la práctica

6.1 Del código C a instrucciones

El compilador traduce el comportamiento del programa, no cada línea de C de forma independiente. Una variable puede residir en un registro, en la pila o desaparecer por optimización. Un bucle puede conservar un salto hacia atrás, transformarse o calcularse en tiempo de compilación. Por eso se debe explicar el **efecto** de cada fragmento, evitando reconstruir un C literal cuando el binario no lo sostiene.

En x86-64 cada registro tiene vistas de distinto tamaño: por ejemplo, `rax`, `eax`, `ax` y `al`. Escribir en `eax` pone a cero la mitad alta de `rax`. La anchura se obtiene de registros y operandos, no del nombre asignado a la variable.

6.2 Convención System V AMD64

En el Linux de referencia, los seis primeros argumentos enteros o punteros se pasan por `rdi`, `rsi`, `rdx`, `rcx`, `r8` y `r9`. Un entero se devuelve habitualmente en `eax`; un puntero o entero de 64 bits, en `rax`. `rsp` apunta a la pila. `rbp` puede usarse como marco en el build `-O0`, pero el optimizador puede prescindir de él.

`cmp` y `test` actualizan flags. El salto posterior determina la interpretación: `jg/jl` expresan comparaciones con signo y `ja/jb`, sin signo. `je/jne` comprueban igualdad. `call` guarda la dirección de retorno y transfiere el control; `ret` la recupera. `lea` calcula una dirección o expresión y no implica por sí sola una lectura.

6.3 Dirección y contenido

`mov eax,[rbp-4]` lee cuatro bytes; `lea rax,[rbp-4]` obtiene la dirección. En la entrega se separan ubicación, bytes leídos/escritos, anchura e interpretación:

Elemento	Pregunta
Dirección	¿Dónde está el dato?
Contenido	¿Qué bytes se leen o escriben?
Anchura	¿Cuántos bytes participan?
Interpretación	¿Entero con/sin signo, puntero u otra posibilidad?

7 Software requerido y comprobación previa

Obligatorio: Linux x86-64, GCC, GNU Make, `objdump`, `file`, `sha256sum` y un editor. No se necesita GDB ni Binary Ninja para el núcleo.

```
bash PreparaciónEntorno/entorno/provision.sh --check
gcc --version | head -n 1
objdump --version | head -n 1
uname -m
```

La ampliación AArch64 solo se hace si el docente ha preparado el compilador y `aarch64-linux-gnu-objdump`.

8 Guía detallada de realización

8.1 1. Preparar y registrar las muestras

```
make -C muestras/build x86
mkdir -p resultados/P1
sha256sum muestras/build/salida/ejemplos-x86-dbg \
  muestras/build/salida/ejemplos-x86-opt \
  > resultados/P1/muestras.sha256
file muestras/build/salida/ejemplos-x86-dbg \
  muestras/build/salida/ejemplos-x86-opt \
  > resultados/P1/file.txt
```

Anote arquitectura, compilador, flags del Makefile y hashes. Los hashes identifican este build; no se copian de otra máquina.

8.2 2. Predecir antes de mirar el ensamblador

Trace en papel `funcion_bucle` para `n=0`, `1` y `10`: valor inicial, condición, iteraciones y retorno. Trace las dos llamadas de `funcion_llamada_con_argumentos(1,2,3)`. Guarde la predicción antes de ejecutar.

8.3 3. Obtener un desensamblado estable

```
objdump -d -M intel --disassemble=funcion_bucle \
  muestras/build/salida/ejemplos-x86-dbg \
  > resultados/P1/bucle-O0.asm
objdump -d -M intel --disassemble=funcion_bucle \
  muestras/build/salida/ejemplos-x86-opt \
  > resultados/P1/bucle-O2.asm
objdump -d -M intel --disassemble=funcion_llamada_con_argumentos \
  muestras/build/salida/ejemplos-x86-dbg \
  > resultados/P1/llamada-O0.asm
```

En `-O0`, marque inicialización, condición, cuerpo, incremento, retorno y accesos a variables locales. Para cada instrucción elegida anote dirección, bytes, operandos y efecto. Explique qué indicio confirma tamaño y signo.

8.4 4. Reconstruir el flujo

Dibuje bloques básicos: un bloque termina antes de un salto, retorno o destino de salto. Para el bucle identifique estado inicial, comparación, acumulación, incremento y retorno a la condición. Si el optimizador elimina alguna forma, describa la transformación real en lugar de inventar instrucciones ausentes.

8.5 5. Comparar `-O0` y `-O2`

Compare accesos a pila, uso de registros, llamadas conservadas o integradas y forma de las condiciones. Elija dos transformaciones y explique por qué mantienen el resultado para las entradas estudiadas. No generalice el resultado a todo `-O2`.

8.6 6. Contrastar las predicciones

```
muestras/build/salida/ejemplos-x86-dbg R3v3rs1ng_UAL \
  > resultados/P1/ejecucion.txt
printf 'codigo_salida=%s\n' "$?" >> resultados/P1/ejecucion.txt
```

La ejecución es un control final. Si contradice la predicción, conserve ambas y localice la premisa incorrecta.

9 Controles de calidad y errores frecuentes

- No confundir una dirección impresa a la izquierda con un valor procesado.
- No atribuir a `cmp` una comparación con signo sin mirar el salto consumidor.
- No exigir prólogo con `push rbp` en código optimizado.
- No contar instrucciones como medida directa de velocidad.
- No usar direcciones de otra compilación.
- No presentar la salida de ejecución como explicación del algoritmo.

10 Estructura recomendada de la entrega

1. Identificación de muestras y entorno.
2. Predicciones previas.
3. Tabla dirección → instrucción → efecto para `-O0`.
4. Grafo o esquema del bucle y de una llamada.
5. Comparación razonada `-O0`/`-O2`.
6. Contraste con ejecución y límites.
7. Anexo con comandos y salidas completas.

Localizador	**Bytes/instrucción**	**Estado leído**	**Estado escrito**	**Interpretación**
dirección del build	completar	completar	completar	justificar